

Análisis del componente servidor HTTP

A continuación se presenta la documentación detallada y estructurada en bloques lógicos del componente de servidor HTTP. Se ha priorizado la explicación de las decisiones arquitectónicas de bajo nivel y el flujo de memoria.

1. Contrato Base: i_http_server.hpp

```
#ifndef I_HTTP_SERVER_HPP
#define I_HTTP_SERVER_HPP

#include "i_component.hpp"
#include "http_types.hpp"
#include <functional>
#include <string>

using RouteHandler = std::function<HttpResponse(const HttpRequest&)>;

class IHttpServer : public IComponent
{
public:
    virtual ~IHttpServer() noexcept = default;
    virtual ComponentResult start(int port) noexcept = 0;
    virtual void stop() noexcept = 0;
    virtual void add_route(const std::string& method, const std::string& path,
RouteHandler handler) = 0;

    // Fachadas semánticas (Sugar Syntax)
    virtual void get(const std::string& path, RouteHandler h) = 0;
    virtual void post(const std::string& path, RouteHandler h) = 0;
    virtual void put(const std::string& path, RouteHandler h) = 0;
    virtual void del(const std::string& path, RouteHandler h) = 0;
    virtual void patch(const std::string& path, RouteHandler h) = 0;
    virtual void serve_static(const std::string& route, const std::string& filepath,
const std::string& content_type) = 0;
};

#endif // I_HTTP_SERVER_HPP
```

Este archivo define la interfaz virtual pura que el Host (la aplicación principal) utilizará para interactuar con el componente sin conocer su implementación subyacente.

- **using RouteHandler = std::function<HttpResponse(const HttpRequest&>>**: Define el tipo de dato para los *callbacks* de enrutamiento. Exige que toda función que maneje una ruta reciba un DTO constante (HttpRequest) y retorne un DTO por valor (HttpResponse).
- **virtual ComponentResult start(int port) noexcept = 0;**: Inicia el ciclo de vida del servidor. El uso de `noexcept` prohíbe explícitamente que las excepciones de red (ej. fallos al abrir sockets) crucen la frontera del ABI. Todo debe resolverse en un código de retorno `ComponentResult`.
- **Fachadas Semánticas (get, post, etc.)**: Proveen "azúcar sintáctico" (Syntactic Sugar) para registrar manejadores sin tener que pasar el verbo HTTP como un string en cada llamada.

2. Modelos de Transferencia de Datos: http_types.hpp

Contiene los Data Transfer Objects (DTO) responsables de transportar la información entre la red, la infraestructura y la lógica de negocio.

HttpRequest

- **std::unordered_map<std::string, std::string> headers**: Almacena las cabeceras HTTP de forma asociativa para búsqueda rápida con complejidad $O(1)$.
- **std::vector<char> body**: Decisión crítica de diseño. En lugar de usar `std::string`, se utiliza un vector de bytes crudos (`char`). Esto garantiza que el servidor sea "Binary-Safe", permitiendo la recepción de archivos PDF, imágenes o audio sin corromper los datos al encontrar caracteres nulos (`\0`).

HttpResponse (Patrón Builder)

- **HttpResponse& set_status(HttpStatusCode code)**: Retorna una referencia a la propia instancia (`*this`), lo que permite encadenar métodos de forma fluida: `res.set_status(...).set_body(...)`.

- **set_file(const std::string& filepath, const std::string& content_type, size_t file_size):** Prepara la respuesta para una transferencia eficiente (*Zero-Copy*). Registra la ruta en `file_path_` e inyecta las cabeceras `Content-Type` y `Content-Length` en el mapa de cabeceras, sin cargar un solo byte del archivo en la memoria de la aplicación.

3. Utilidades Estáticas: `http_parser.hpp`

Este archivo contiene la lógica pura de procesamiento de cadenas.

- **url_decode:** Iterador de caracteres. Cuando encuentra un `%`, extrae los siguientes dos caracteres, los procesa a través de un `std::istream` en modo hexadecimal (`std::hex`) y los convierte a su equivalente ASCII (`static_cast<char>`). Vital para decodificar espacios (`%20`) o caracteres especiales en las URL.
- **parse_headers:** Recorre el bloque de cabeceras línea por línea. Separa la clave del valor utilizando los dos puntos (`:`) como delimitador. Aplica un trim básico eliminando los espacios sobrantes al inicio del valor (`value.erase(...)`).

4. Definición de la Infraestructura: `http_server_component.hpp`

Define la clase concreta y encapsula los recursos del sistema operativo.

- **class SocketRAII:** Implementa el patrón RAII (*Resource Acquisition Is Initialization*). Toma posesión de un descriptor de archivo nativo (un entero `fd_`) en su constructor. Su destructor invoca automáticamente `close(fd_)`. Esto garantiza que los sockets de los clientes se cierren de manera determinista al salir del ámbito de ejecución, eliminando fugas de descriptores de archivo (*file descriptor leaks*).
- **std::unordered_map<std::string, std::unordered_map<std::string, RouteHandler>> routes_;** Estructura de enrutamiento anidada. El mapa exterior indexa por verbo HTTP (ej. "GET"), y el mapa interior indexa por ruta (ej. "/api/contenidos").
- **std::atomic<bool> is_running_{false};** Variable booleana thread-safe. Garantiza que múltiples hilos (el hilo principal y el hilo del servidor) puedan leer y modificar el estado de ejecución sin incurrir en condiciones de carrera.

5. Lógica de Ejecución: `http_server_component.cpp`

Este es el núcleo operativo del componente, donde se interactúa directamente con la API POSIX de Linux/Unix.

Ciclo de Vida y Configuración del Socket

- `server_fd_ = socket(AF_INET, SOCK_STREAM, 0);`: Solicita al kernel un descriptor de archivo para un socket IPv4 (`AF_INET`) de tipo TCP (`SOCK_STREAM`).
- `setsockopt(..., SO_REUSEADDR, ...)`: Opción crítica en el desarrollo de servidores. Indica al kernel que libere inmediatamente el puerto si el servidor se reinicia, evitando el error `Address already in use` cuando el socket aún permanece en estado `TIME_WAIT`.
- `std::thread(&HttpServerComponent::listen_and_serve, this, port);`: Desacopla la ejecución. El hilo principal del Host regresa inmediatamente a `main()`, mientras el servidor entra en su bucle infinito en un subproceso.
- `shutdown(server_fd_, SHUT_RDWR);` (dentro de `stop`): Interrumpe abruptamente las operaciones de lectura y escritura del socket principal. Esto obliga a la función bloqueante `accept()` a fallar inmediatamente y salir del bucle de espera, permitiendo un apagado limpio (*graceful shutdown*).

Bucle de Aceptación (`listen_and_serve`)

- `int client_fd = accept(...)`: El hilo se suspende aquí pasivamente hasta que un cliente web intenta conectarse. Al ocurrir, el kernel despierta al hilo y devuelve un nuevo descriptor de archivo (`client_fd`) exclusivo para hablar con ese cliente.
- `std::thread(&HttpServerComponent::process_client, this, client_fd).detach();`: Por cada cliente conectado, se lanza un hilo independiente. `.detach()` separa el hilo del control de la aplicación principal, permitiendo que viva por su cuenta y se autodestruya al terminar de procesar la petición.

Procesamiento de la Petición (process_client)

- **Etapa 1 (Lectura de Cabeceras):** Lee bloques de 1024 bytes del socket hasta que detecta la secuencia de terminación del protocolo HTTP estándar: `\r\n\r\n`. Todo lo anterior conforma la línea de control (Verbo y URL) y las cabeceras.
- **Etapa 2 (Lectura del Cuerpo):** Extrae la cabecera Content-Length. Si el tamaño del cuerpo es mayor a los bytes sobrantes leídos en la primera etapa, entra en un bucle while iterativo llamando a `read(...)` hasta consumir exactamente los bytes declarados. Esto previene que el servidor se quede bloqueado esperando datos infinitamente o que lea datos incompletos.
- **Etapa 3 (Enrutamiento Dinámico):** Busca una coincidencia exacta de la ruta en el mapa `routes_`. Si falla, itera sobre las rutas registradas buscando comodines (*) al final. Compara prefijos utilizando `req.path.find(prefix) == 0`.
- **Etapa 4 (Envío de Cabeceras):** Serializa las cabeceras generadas en el objeto `HttpResponse` a una cadena de texto plana y las envía por el socket al cliente mediante la función `send()`.
- **Etapa 5 (Zero-Copy Transfer):** Si se configuró un archivo físico, se abre un descriptor de solo lectura (`O_RDONLY`). Se invoca `sendfile(...)`, lo que ordena al DMA (*Direct Memory Access*) del hardware y al kernel mover los bloques del disco directamente a la tarjeta de red, evadiendo la copia hacia el espacio de usuario (RAM de tu programa).
- **Etapa 6 (Logueo Seguro):** `std::lock_guard<std::mutex> lock(console_mutex_)` toma el control de un candado de exclusión mutua. Como hay múltiples hilos procesando clientes al mismo tiempo, esto evita que los textos en `std::cout` se sobrescriban e intercalen de forma ilegible en la terminal. Al salir del ámbito de la función, el `lock_guard` se destruye y libera el candado automáticamente.